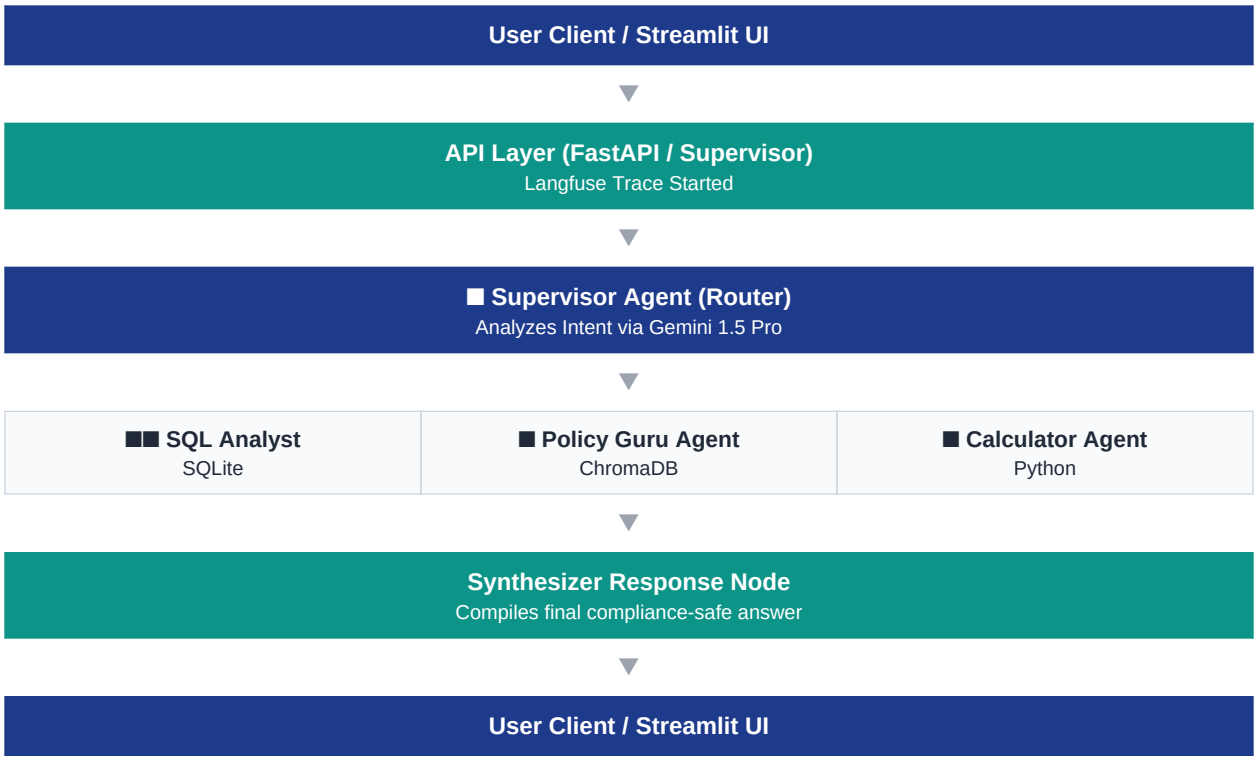
Loan Navigator Agent Suite

Supplementary Documentation

This document serves as the final technical artifact for the Loan Navigator Agent Suite, fulfilling the requirements for API specifications, interaction diagrams, prompt templates, and the complete setup runbook.

1. Agent Interaction Diagram

The following diagram illustrates the flow of data and execution state between the user, the orchestration layer (Supervisor), the specialized worker agents, and the final response synthesizer.



2. OpenAPI Specification

The following is the OpenAPI 3.0 specification for the FastAPI wrapper that exposes the LangGraph multi-agent suite.

```
openapi: 3.0.2
info:
  title: Loan Navigator Agent Suite API
  description: Multi-agent system for resolving loan queries using LangGraph and Vertex AI.
  version: 1.0.0
paths:
  /health:
    get:
      summary: Health Check
      operationId: health_check_health_get
      responses:
        '200':
          description: Successful Response
          content:
            application/json:
              schema:
                type: object
                properties:
                  status:
                    type: string
                    example: healthy
  /api/v1/query:
    post:
      summary: Process Query
      description: Endpoint to process natural language queries through the Supervisor Agent.
      operationId: process_query_api_v1_query_post
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/QueryRequest'
        required: true
      responses:
        '200':
          description: Successful Response
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/QueryResponse'
        '500':
          description: Internal Server Error
```

```
components:
  schemas:
    QueryRequest:
      title: QueryRequest
      required:
        - query
      type: object
      properties:
        query:
          title: Query
          type: string
        user_id:
          title: User Id
          type: string
          default: anonymous
    QueryResponse:
      title: QueryResponse
      required:
        - response
        - status
      type: object
      properties:
        response:
          title: Response
          type: string
        status:
          title: Status
          type: string
```

3. Prompt Templates

The behavior, safety, and accuracy of the system are governed by strictly engineered prompt templates injected at various nodes of the LangGraph execution.

Supervisor Agent (Router) Prompt

```
You are an expert routing assistant for a loan processing system. Your job is to determine
the best agent to handle a user's query.
The available agents are:
- 'sql_agent': Use for questions about specific loan details like "what is my balance?",
"show my EMI amount", or any query that requires fetching exact data for a specific loan from
a database.
- 'policy_agent': Use for general questions about company rules, prepayment policies, top-up
eligibility criteria, or regulatory guidelines.
- 'calculator_agent': Use for "what-if" scenarios, such as "what happens if I prepay
50,000?", or any query that requires mathematical calculation.
- 'end_conversation': Use for simple greetings, thank yous, or any query that does not
require using a tool.
```

SQL Analyst Agent Prompt

```
You are a SQL writer. Your only job is to write a valid SQL query for a database.
The database contains one table named 'loan_data'.

SCHEMA:
- "loan_id" INTEGER
- "customer_id" INTEGER
- "loan_amount" REAL
- "amount_paid" REAL
- "status" TEXT
- "topup_eligible" INTEGER

RULES:
- To calculate 'outstanding balance', use the formula: (loan_amount - amount_paid).
- If a user provides a loan ID like 'LN2003', use only the integer part (2003) in the WHERE
clause.
- Output ONLY the raw SQL query. Do not add explanations, markdown, or the word 'SQLite'.
```

Policy Guru Agent (RAG) Prompt

```
You are the Chief Compliance & Policy Advisor for BlueLoans4all.
Answer the user's question accurately based ONLY on the provided policy context below.

Rules:
- Ground your response STRICTLY in the context provided.
- Cite the specific PDF document name and page number when explaining rules.
- If the context does not contain enough information, state clearly that the policy manuals
do not cover this specific edge case.
- Format your response cleanly using bullet points if listing out eligibility criteria or
steps.

POLICY CONTEXT:
{context}
```

What-If Calculator Agent Prompt

```
You are a data extraction assistant. Extract the financial parameters from the user's query
to feed into a prepayment calculator.

If a value is missing from the user's query, assume the following defaults:
- principal: 100000 (if not specified)
- interest_rate: 10.0 (if not specified)
- tenure_months: 24 (if not specified)
- prepayment_amount: 10000 (if not specified)

Output MUST be a valid JSON matching the required schema.
```

Synthesizer Agent Prompt

```
You are a helpful and friendly AI assistant for BlueLoans4all, an Indian financial company.
Your task is to create a single, clear, and concise response for the user based on the
information gathered by our internal tools.
Combine the information from the different sources into a single, easy-to-understand answer.

- Address the user's original question directly.
- Crucially, when mentioning any monetary values (like loan balances, EMIs, or amounts), you
MUST use the Indian Rupee symbol (■).
- Do not just list the raw data. Explain what it means in a helpful way.
- Maintain a professional and helpful tone.
- Do not mention the internal tools or agent names (e.g., "The SQL agent found..."). Just
present the final answer.
```

4. Setup & Deployment Runbook

Phase 1: Local Environment Setup

1. **Clone Repository:**

```
git clone <repository_url> && cd Loan-Navigator-Agent-Suite
```

2. **Create Virtual Environment:**

```
python -m venv .venv && source .venv/bin/activate
```

3. **Install Dependencies:**

```
pip install -r app/requirements.txt
```

4. **Configure Secrets (.env):** Create app/.env with the following:

```
GOOGLE_API_KEY=AIza...
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_SECRET_KEY=sk-lf-...
LANGFUSE_HOST=https://cloud.langfuse.com
GCP_PROJECT_ID=bdc-trainings
GCP_LOCATION=us-central1
GEMINI_MODEL=gemini-2.5-flash
EMBEDDING_MODEL=models/gemini-embedding-001
GOOGLE_GENAI_USE_VERTEXAI=True
```

5. **Ingest Vector Data:** Ensure PDF policies are in data/policy_docs/. Run:

```
python ingest.py
```

6. **Local Testing:** Launch the Streamlit UI:

```
streamlit run streamlit_app.py
```

Phase 2: Google Cloud Deployment

1. **Authenticate and Set Project:**

```
gcloud auth login
gcloud config set project <YOUR_PROJECT_ID>
```

2. **Build Container & Push to Artifact Registry:**

```
gcloud builds submit --tag
us-central1-docker.pkg.dev/<YOUR_PROJECT_ID>/loan-navigator/app:latest .
```

3. **Create Google Secret Manager Vaults:** Create secrets for google-api-key, langfuse-public-key, and langfuse-secret-key via the GCP Console.

4. **Grant Cloud Run Access to Secrets:** Ensure the Compute Engine default service account (or the specific Cloud Run service identity) has the Secret Manager Secret Accessor role.

5. **Deploy to Cloud Run:**

```
gcloud run deploy loan-navigator-app \
  --image us-central1-docker.pkg.dev/<YOUR_PROJECT_ID>/loan-navigator/app:latest \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --set-env-vars="GOOGLE_GENAI_USE_VERTEXAI=True,GEMINI_MODEL=gemini-2.5-flash,EMBEDDING_MODE
L=models/gemini-embedding-001,LANGFUSE_HOST=https://cloud.langfuse.com,GCP_PROJECT_ID=<YOUR_P
ROJECT_ID>,GCP_LOCATION=us-central1" \
  --update-secrets="GOOGLE_API_KEY=google-api-key:latest,LANGFUSE_PUBLIC_KEY=langfuse-public-
key:latest,LANGFUSE_SECRET_KEY=langfuse-secret-key:latest"
```

Phase 3: Observability Setup

1. **Langfuse:** Log into Langfuse to view active AI traces linked to the session runs.

2. **Google Cloud Monitoring:** Navigate to Metrics Explorer in GCP. Search for the custom metric custom.googleapis.com/agent/invocation_count and custom.googleapis.com/agent/fallback_count. Group by agent_name to create operational dashboards.